41039 Programming 1

Revision of Week 9: exceptions and reading text



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Today's revision: 10 questions, about 35 minutes

#	Type	Topic
Q1	Fill in the blanks	Checked, `finally`, end of input
Q2	Multiple choice	Which `main` compiles?
Q3	Multiple choice	Which loops read every line?
Q4	Predict the output	Marks through calls (Java)
Q5	Predict the output	A file of marks (Python)
Q6	Find the bug	The mark reader (Java, 4 bugs)
Q7	Find the bug	Average of a file (Python, 3 bugs)
Q8	Write the code	Parse a mark or say why (Java)
Q9	Write the code	Ask again until it is valid (Python)
Q10	Write the code	Add up the numbers (Java)

How it works

1. Read the question and answer it **alone** first. Write your answer down.
2. Then compare with a neighbour.
3. The answer appears on the **next click**.

Week 9 in five ideas (read this first)

1. **An exception jumps.** A failing statement skips the rest of the `try`; control goes to the first matching `catch` / `except`. No match: the program stops with a *stack trace*.
2. **Catch what you expect.** `Integer.parseInt` throws `NumberFormatException`; Python's `int()` raises `ValueError`. Python's `else:` runs only if nothing failed.
3. **Checked or unchecked.** `IOException` is *checked*: catch it or declare `throws`. Others, like `NumberFormatException`, are unchecked.
4. **Throw and clean up.** `throw` / `raise` reports a broken rule to the caller. `finally` runs on success **and** on failure. try-with-resources and `with` close a reader for you.
5. **Know the reading contract.** `Scanner` reads tokens, skipping spaces and newlines. `readLine()`: `""` is a blank line, `null` is the end. Python `readline()`: `"\n"` is a blank line, `""` is the end.

```java
String line;
while ((line = in.readLine()) != null)
    System.out.println(line);  // "" if blank
```

Q1 Fill in the blanks Exceptions and end of input

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- `IOException` must be caught or declared: it is **(1)** . `NumberFormatException` need not be: it is **(2)** .
- A **(3)** block runs whether the `try` succeeded or failed.
- Python's **(4)** block closes the file when it ends.
- At the end of input, `readLine()` returns **(5)** . A blank line gives an **(6)** string.

Word bank (2 min, alone)

a) checked	**f)** empty
b) unchecked	**g)** else
c) finally	**h)** ready
d) with	**i)** throws
e) null	**j)** catch

Q1 Fill in the blanks Exceptions and end of input

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- `IOException` must be caught or declared: it is **(1)** . `NumberFormatException` need not be: it is **(2)** .
- A **(3)** block runs whether the `try` succeeded or failed.
- Python's **(4)** block closes the file when it ends.
- At the end of input, `readLine()` returns **(5)** . A blank line gives an **(6)** string.

Answers

(1) a) checked **(2)** b) unchecked
(3) c) finally
(4) d) with
(5) e) null **(6)** f) empty
Not used: g) else, h) ready, i) throws, j) catch.

`catch` runs only on failure, Python's `else` only on success, `finally` on both. `ready()` is **not** an end test: loop until `null`.

Q2 Multiple choice Which `main` compiles?

```
1  import java.io.*;
2  public class Demo {
3      static String firstLine(String path) throws IOException {
4          try (BufferedReader in = new BufferedReader(new FileReader(path))) {
5              return in.readLine();
6          }
7      }
8      public static void main(String[] args) {
9          System.out.println(firstLine("notes.txt"));
10     }
11 }
```

Only one compiles. Which? (90 s)

A. `main` as shown

B. catch `NumberFormatException` around line 9

C. add `throws IOException` to line 8

D. `try/finally` around line 9

Hint: `firstLine` never changes. For each `main`, ask: is the `IOException` that `firstLine` may throw **caught**, or **declared**?

Q2 Multiple choice Which `main` compiles?

```
1  import java.io.*;
2  public class Demo {
3      static String firstLine(String path) throws IOException {
4          try (BufferedReader in = new BufferedReader(new FileReader(path))) {
5              return in.readLine();
6          }
7      }
8      public static void main(String[] args) {
9          System.out.println(firstLine("notes.txt"));
10     }
11 }
```

Only one compiles. Which? (90 s)

A. `main` as shown

B. catch `NumberFormatException` around line 9

✓ **C.** add `throws IOException` to line 8

D. `try/finally` around line 9

Answer: C

`IOException` is **checked**: catch it or declare it. A, B, D give `unreported exception IOException; must be caught or declared to be thrown`. B catches the wrong type; `finally` catches nothing. C declares it, so `main` passes it on.

Q3 Multiple choice Which loops read every line?

Goal: print every line of `in`, a `BufferedReader` over `"tea\n\ncoffee"` (middle line blank, no newline at the end), then stop.

Version 1

```
String line;
while ((line = in.readLine())
        != null) {
    System.out.println(line);
}
```

Version 2

```
String line = in.readLine();
while (!line.isEmpty()) {
    System.out.println(line);
    line = in.readLine();
}
```

Version 3

```
String line = in.readLine();
while (line != null) {
    System.out.println(line);
    line = in.readLine();
}
```

Choose one (90 seconds)

A. Version 1 only
B. Versions 1 and 3
C. Versions 2 and 3
D. All three

Hint: all three compile. Write down what each `readLine()` call returns: three lines, then the end.

Q3 Multiple choice Which loops read every line?

Goal: print every line of `in`, a `BufferedReader` over `"tea\n\ncoffee"` (middle line blank, no newline at the end), then stop.

Version 1

```
String line;
while ((line = in.readLine())
        != null) {
    System.out.println(line);
}
```

Version 2

```
String line = in.readLine();
while (!line.isEmpty()) {
    System.out.println(line);
    line = in.readLine();
}
```

Version 3

```
String line = in.readLine();
while (line != null) {
    System.out.println(line);
    line = in.readLine();
}
```

Choose one (90 seconds)

- **A.** Version 1 only
- ✓ **B.** Versions 1 and 3
- **C.** Versions 2 and 3
- **D.** All three

Answer: B

V1 ✓ `tea`, empty line, `coffee`: `null` comes only after `coffee`, newline or not.
V2 ✗ takes the blank line (`""`) for the end: only `tea`.
V3 ✓ same as V1, with the read written twice.

Q4 Predict the output Marks through method calls

```
static int mark(String s) {
    int m = Integer.parseInt(s);
    if (m > 100)
        throw new IllegalArgumentException("too big");
    return m;
}
static int total(String[] marks) {
    int sum = 0;
    for (String s : marks) {
        try {
            sum += mark(s);
        } catch (NumberFormatException e) {
            System.out.println("skip " + s);
        } finally {
            System.out.println("checked " + s);
        }
    }
    return sum;
}
```

All `static`, in class `Demo`. In `main`:

```
String[] a = {"40", "x"};
String[] b = {"120", "9"};
try {
  System.out.println(total(a));
  System.out.println(total(b));
} catch (IllegalArgumentException e) {
  System.out.println("main: " + e.getMessage());
}
```

Predict (90 seconds, alone)

1. Write down the **six** lines that are printed.

2. Add one `catch` to `total` so that `"120"` is skipped like `"x"`. What does the second call print then?

Q4 Predict the output Marks through method calls

```
static int mark(String s) {
    int m = Integer.parseInt(s);
    if (m > 100)
        throw new IllegalArgumentException("too big");
    return m;
}
static int total(String[] marks) {
    int sum = 0;
    for (String s : marks) {
        try {
            sum += mark(s);
        } catch (NumberFormatException e) {
            System.out.println("skip " + s);
        } finally {
            System.out.println("checked " + s);
        }
    }
    return sum;
}
```

All `static`, in class `Demo`. In `main`:

```
String[] a = {"40", "x"};
String[] b = {"120", "9"};
try {
  System.out.println(total(a));
  System.out.println(total(b));
} catch (IllegalArgumentException e) {
  System.out.println("main: " + e.getMessage());
}
```

Answers

1. `checked 40  skip x  checked x  40  checked 120  main: too big`

`"120"` is not a `NumberFormatException`. `finally` prints, then it leaves `total` for `main`'s `catch`: `"9"` is never read.

2. Add `catch (IllegalArgumentException e)` printing `"skip " + s`. Then: `skip 120, checked 120, checked 9, 9`.

Q5 Predict the output A file of marks

```
with open("marks.txt", "w", encoding="utf-8") as f:
    f.write("Linh,85\n\nAn,abc\nMai,70")

count = 0
total = 0
with open("marks.txt", encoding="utf-8") as f:
    for line in f:
        count += 1
        try:
            name, mark = line.strip().split(",")
            total += int(mark)
        except ValueError:
            print("line", count, "skipped")
        else:
            print(name, "ok")
print(count, total)
```

Lines 1–2 make the input file: 4 lines, the second one blank.

Predict (90 seconds, alone)

1. Write down the **five** lines that are printed.

2. File lines 2 and 3 both raise `ValueError`, but in different statements. Which, and why?

Q5 Predict the output A file of marks

```
with open("marks.txt", "w", encoding="utf-8") as f:
    f.write("Linh,85\n\nAn,abc\nMai,70")

count = 0
total = 0
with open("marks.txt", encoding="utf-8") as f:
    for line in f:
        count += 1
        try:
            name, mark = line.strip().split(",")
            total += int(mark)
        except ValueError:
            print("line", count, "skipped")
        else:
            print(name, "ok")
print(count, total)
```

Lines 1–2 make the input file: 4 lines, the second one blank.

Answers

1. `Linh ok`
`line 2 skipped`
`line 3 skipped`
`Mai ok`
`4 155`

2. Blank line: `"".split(",")` is `['']`, one piece for two names, so `name, mark = ...` fails (`not enough values to unpack`). `An,abc` splits fine, then `int("abc")` fails.

`Mai,70` has no `"\n"` but is still read.

Q6 Find the bug The mark reader (Java)

```java
import java.io.*;
public class MarkReader {
    static int sumMarks(BufferedReader in) {
        int total = 0, n = 0;
        String line;
        try {
            while ((line = in.readLine()) != null) {
                if (line.isEmpty()) break;
                n++;
                total += Integer.parseInt(line);
            }
        } catch (NumberFormatException e) {
            System.out.println("bad line " + n);
        }
        return total;
    }
}
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

What the method should do

- One mark per line; read to the end
- Skip blank lines, but count them (from 1)
- Not a number: print bad `line N`, **go on**
- Return the total of the good marks

Test (`main` in `Demo`, with `throws IOException`)

```java
String text = "50\n\nabc\n30\nxyz\n20";
BufferedReader in = new BufferedReader(
    new StringReader(text));
System.out.println(MarkReader.sumMarks(in));
```

Expected output

```
bad line 3
bad line 5
100
```

Q6 Find the bug The mark reader (Java)

```java
import java.io.*;
public class MarkReader {
    static int sumMarks(BufferedReader in) {
        int total = 0, n = 0;
        String line;
        try {
            while ((line = in.readLine()) != null) {
                if (line.isEmpty()) break;
                n++;
                total += Integer.parseInt(line);
            }
        } catch (NumberFormatException e) {
            System.out.println("bad line " + n);
        }
        return total;
    }
}
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors. Give each line number and its fix.

✓ The 4 fixes

- **Line 3** (compile error, reported at line 7: `unreported exception IOException; must be caught or declared to be thrown`): add `throws IOException`.
- **Line 8**: `break` stops at the blank line. Use `continue`.
- **Line 9**: blank lines are not counted. Move `n++` above line 8.
- **Lines 6, 12**: the `try` wraps the whole loop, so the first bad line ends the reading. Put `try`/`catch` around line 10 only.

Fix only line 3 and it prints just 50.

Q7 Find the bug Average of a file (Python)

```
1  def average(path):
2      try:
3          f = open(path)
4      except ValueError:
5          print("no file:", path)
6          return None
7      total, count = 0, 0
8      line = f.readline().strip()
9      while line != "":
10         total += int(line)
11         count += 1
12         line = f.readline().strip()
13     f.close()
14     return total / count
15 with open("nums.txt", "w") as out:
16     out.write("4\n6\n\n8\n")
17 open("empty.txt", "w").close()
18 print(average("nums.txt"))
19 print(average("empty.txt"))
20 print(average("missing.txt"))
```

✗ **Find 3 bugs** in lines 1–14 (3 min). Give each line number and its fix.

What `average` should do

- Return the average of the numbers, one per line
- Skip blank lines. No numbers: return `None`
- No such file: print `no file: <path>`, return `None`

Expected output

```
6.0
None
no file: missing.txt
None
```

✗ Actual output (stack trace trimmed)

```
5.0
Traceback (most recent call last):
  File "main.py", line 19, in <module>
  File "main.py", line 14, in average
ZeroDivisionError: division by zero
```

Q7 Find the bug Average of a file (Python)

```
1  def average(path):
2      try:
3          f = open(path)
4      except ValueError:
5          print("no file:", path)
6          return None
7      total, count = 0, 0
8      line = f.readline().strip()
9      while line != "":
10         total += int(line)
11         count += 1
12         line = f.readline().strip()
13     f.close()
14     return total / count
15 with open("nums.txt", "w") as out:
16     out.write("4\n6\n\n8\n")
17 open("empty.txt", "w").close()
18 print(average("nums.txt"))
19 print(average("empty.txt"))
20 print(average("missing.txt"))
```

✗ **Find 3 bugs** in lines 1–14 (3 min). Give each line number and its fix.

✓ The 3 fixes

- **Line 4**: `open` raises `FileNotFoundError`, not `ValueError`. Catch `FileNotFoundError`.
- **Lines 8, 12**: `strip()` *before* the test makes the blank line `""`, which looks like the end: `8` is never read. Use `for line in f:`, and `continue` if `line.strip() == ""`.
- **Line 14**: no numbers, so `count` is 0. First: `if count == 0: return None`.

Read a Python trace from the **bottom**: the error, where it happened (line 14), who called (line 19).

Q8 Write the code Parse a mark or say why (Java)

Finish the `TODO` (alone, 4 min). `trim()` first, as `Integer.parseInt(" 85")` throws.

- Not a whole number: throw `IllegalArgumentException("not a number: <trimmed text>")`.
- Below 0 or above 100: throw `IllegalArgumentException("out of range: <mark>")`.

Skeleton

```
class Marks {
    // " 85" gives 85
    static int parseMark(String text) {
        // TODO
        return 0;
    }
}
```

Test (in `main`)

```
String[] inputs = {" 85", "abc", "120", "0"};
for (String s : inputs) {
    try {
        System.out.println(Marks.parseMark(s));
    } catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());
    }
}
```

Must print

```
85
not a number: abc
out of range: 120
0
```

Q8 Write the code Parse a mark or say why (Java)

Finish the `TODO` (alone, 4 min). `trim()` first, as `Integer.parseInt(" 85")` throws.

- Not a whole number: throw `IllegalArgumentException("not a number: <trimmed text>")`.
- Below 0 or above 100: throw `IllegalArgumentException("out of range: <mark>")`.

Skeleton

```
class Marks {
    // " 85" gives 85
    static int parseMark(String text) {
        // TODO
        return 0;
    }
}
```

✓ One answer

```
class Marks {
    static int parseMark(String text) {
        String t = text.trim();
        int m;
        try {
            m = Integer.parseInt(t);
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException(
                "not a number: " + t);
        }
        if (m < 0 || m > 100) throw new
            IllegalArgumentException("out of range: " + m);
        return m;
    }
}
```

Q9 Write the code Ask again until it is valid (Python)

Finish the `TODO` (alone, 4 min) so that the test prints the output shown.

- Read lines with `input()`. A whole number from `low` to `high`: return it.
- Otherwise print `not a number: <text>` or `out of range: <n>`, and read the next line.
- At the end of the input, `input()` raises `EOFError`: return `None`.

Skeleton

```
def ask_number(low, high):
    # TODO
    return None
```

Input (typed, or piped in)

```
abc
42
7
x
```

…then the input ends.

Test

```
print(ask_number(1, 10))
print(ask_number(1, 10))
```

Must print

```
not a number: abc
out of range: 42
7
not a number: x
None
```

Q9 Write the code Ask again until it is valid (Python)

Finish the `TODO` (alone, 4 min) so that the test prints the output shown.

- Read lines with `input()`. A whole number from `low` to `high`: return it.
- Otherwise print `not a number: <text>` or `out of range: <n>`, and read the next line.
- At the end of the input, `input()` raises `EOFError`: return `None`.

Skeleton

```
def ask_number(low, high):
    # TODO
    return None
```

Input (typed, or piped in)

```
abc
42
7
x
```

…then the input ends.

✓ One answer

```
def ask_number(low, high):
    while True:
        try:
            text = input()
        except EOFError:
            return None
        try:
            n = int(text)
        except ValueError:
            print("not a number:", text)
            continue
        if low <= n <= high:
            return n
        print("out of range:", n)
```

Q10 Write the code Add up the numbers (Java)

Finish the `TODO` (alone, 4 min) so that the test prints the output shown.

- Read tokens (words between spaces or newlines) while `in.hasNext()`.
- A whole number (`in.hasNextInt()`): add `in.nextInt()`.
- Any other word: print `skip <word>`, reading it with `in.next()` (`nextInt()` would throw `InputMismatchException`).

Skeleton

```
class Tokens {
    static int sumInts(Scanner in) {
        int total = 0;
        // TODO
        return total;
    }
}
```

Test (in `main`)

```
try (Scanner in = new Scanner("10 x 20\n\ny 5")) {
    System.out.println(Tokens.sumInts(in));
}
System.out.println(Tokens.sumInts(new Scanner("")));
```

Must print

```
skip x
skip y
35
0
```

Q10 Write the code Add up the numbers (Java)

Finish the `TODO` (alone, 4 min) so that the test prints the output shown.

- Read tokens (words between spaces or newlines) while `in.hasNext()`.
- A whole number (`in.hasNextInt()`): add `in.nextInt()`.
- Any other word: print `skip <word>`, reading it with `in.next()` (`nextInt()` would throw `InputMismatchException`).

Skeleton

```
class Tokens {
    static int sumInts(Scanner in) {
        int total = 0;
        // TODO
        return total;
    }
}
```

✓ One answer

```
class Tokens {
    static int sumInts(Scanner in) {
        int total = 0;
        while (in.hasNext()) {
            if (in.hasNextInt()) {
                total += in.nextInt();
            } else {
                System.out.println("skip " + in.next());
            }
        }
        return total;
    }
}
```

Looking ahead: from reading marks to saving a report

This week: read, then print

```java
// in reads "Linh,85\nMai,70"
int total = 0, n = 0;
String line;
while ((line = in.readLine()) != null) {
    total += Integer.parseInt(line.split(",")[1]);
    n++;
}
System.out.println(n + " marks, total " + total);
```

Prints `2 marks, total 155`.

Think (1 minute)

The office wants this **saved** in `report.txt`, for another program to read tomorrow. What new things can go wrong?

Looking ahead: from reading marks to saving a report

This week: read, then print

```java
// in reads "Linh,85\nMai,70"
int total = 0, n = 0;
String line;
while ((line = in.readLine()) != null) {
    total += Integer.parseInt(line.split(",")[1]);
    n++;
}
System.out.println(n + " marks, total " + total);
```

Prints `2 marks, total 155`.

Think (1 minute)

The office wants this **saved** in `report.txt`, for another program to read tomorrow. What new things can go wrong?

Opening for writing **replaces** an old file. The text is only safe once the file is **closed**. The format must be one the next program can split.

```java
PrintWriter out =
    new PrintWriter("report.txt", "UTF-8");
out.println("marks," + n);
out.println("total," + total);
// read it back now: 0 lines (still buffered)
out.close();
// read it back now: 2 lines
```

Python's `f.write(...)` adds **no** newline of its own. Today: **writing files and reliable reports**.

You should now be able to...

Exceptions

- ✓ trace what is skipped and what runs (Q4–Q6)
- ✓ catch the exact exception (Q6, Q7, Q8)
- ✓ checked or unchecked? `throws` (Q1, Q2, Q6)
- ✓ `throw` / `raise` with a clear message, clean up with `finally` (Q1, Q4, Q8)
- ✓ read a Python stack trace (Q7)

Reading text

- ✓ read lines until `null`, not a blank line (Q1, Q3, Q6)
- ✓ Python: test for the end **before** you strip (Q5, Q7)
- ✓ read tokens: `hasNextInt()` first (Q10)
- ✓ go on after bad input; stop at the end (Q5, Q6, Q9)
- ✓ close readers: try-with-resources, `with` (Q1, Q2)

Next: Week 10 – Writing files and reliable reports

`PrintWriter` and `write`, overwrite versus append, closing before you check, delimited records (CSV), and testing a report in layers.